

INTELIGENCIA ARTIFICIAL (1INF24)



UNIDAD 1: Introducción a la IA. Búsqueda y optimización en IA

*Tema 2: La Búsqueda dentro de la Inteligencia Artificial
(Parte 2)*

MSc. Luis Muroya

Adaptado de material: Dr. Edwin Villanueva

Contenido

- Búsqueda con Información
 - Búsqueda codiciosa
 - Búsqueda A^*
 - Heurísticas

Recapitulando parte 1...

- **Agentes de resolución de problemas:** percepción, formulación, solución (búsqueda), acciones.
- **Formulación de problema:** estados, objetivo, prueba-objetivo, acciones, transiciones, costos.
- **Búsqueda sin información:** busca solución sin saber qué tan cerca está un nodo del objetivo.
- Algoritmos (como *tree-search* y *graph-search*):
 - **Búsqueda en Amplitud (BFS):** completo, óptimo, $O(b^d)$ en tiempo y espacio.
 - **Búsqueda en Profundidad (DFS):** completo en espacio finito, sub-óptimo, $O(b^m)$ en tiempo y $O(b^*m)$ en espacio.



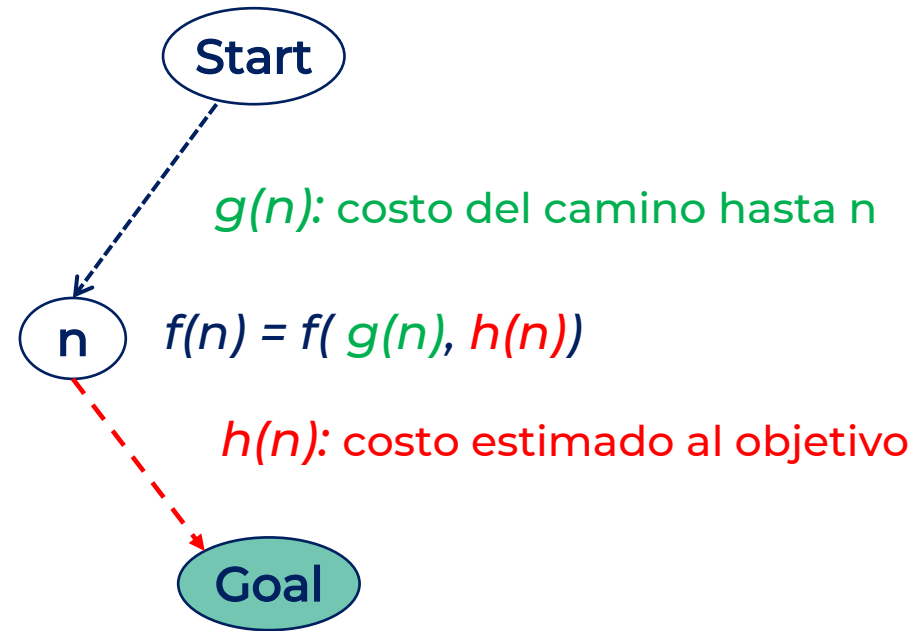
Búsqueda con Información

- Utiliza conocimiento específico sobre el problema para encontrar soluciones de forma mas eficiente que la búsqueda ciega.
 - Conocimiento específico **adicional** a la definición del problema.
- Enfoque general: **búsqueda por la mejor opción**.
 - Utiliza una **función de evaluación** para cada nodo.
 - Expande el nodo que tiene la función de evaluación más baja.
 - **Implementación:** usa cola prioritaria (*priority queue*)



Búsqueda con Información

- **Idea:** usar una función de evaluación $f(n)$ para cada nodo.
 - $f(n)$ es una estimación de cuán deseable es el nodo n
 - Se expande el nodo mas deseable que aún no fue expandido
 - $f(n)$ es normalmente una combinación del **costo de camino $g(n)$** y de una **función de heurística $h(n)$** que mide el costo estimado para llegar al objetivo desde n



Best-First Search

(Búsqueda por Mejor Opción)

Implementación general:

```
function BEST-FIRST-GRAPH-SEARCH(problem, f) returns a solution, or failure  
  
  node  $\leftarrow$  a node with STATE = problem.INITIAL-STATE  
  frontier  $\leftarrow$  a priority queue ordered by f, with node as the only element  
  explored  $\leftarrow$  an empty set  
  loop do  
    if EMPTY?(frontier) then return failure  
    node  $\leftarrow$  POP(frontier) /* chooses the lowest-cost node in frontier */  
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)  
    add node.STATE to explored  
    for each action in problem.ACTIONS(node.STATE) do  
      child  $\leftarrow$  CHILD-NODE(problem, node, action)  
      if child.STATE is not in explored And child.STATE is not in frontier then  
        frontier  $\leftarrow$  INSERT(child, frontier)  
      else if child.STATE is in some frontier node n with  $f(n) > f(child)$  then  
        replace frontier node n with child
```



Best-First Search

(Búsqueda por Mejor Opción)

- A diferencia de BFS y DFS, los nodos de la frontera son ordenados por la función de evaluación $f(n)$, es decir, la frontera es de tipo **cola de prioridad**
- La forma de $f(n)$ determina la estrategia de búsqueda:
 - Si $f(n) = g(n)$ se tiene **Búsqueda de Costo Uniforme (Uniform Cost)**
 - Si $f(n) = h(n)$ se tiene **Búsqueda Voraz (Greedy)**
 - Si $f(n) = g(n) + h(n)$ se tiene **Búsqueda A* (A-star)**



Greedy Best-First Search

(Búsqueda Voraz)

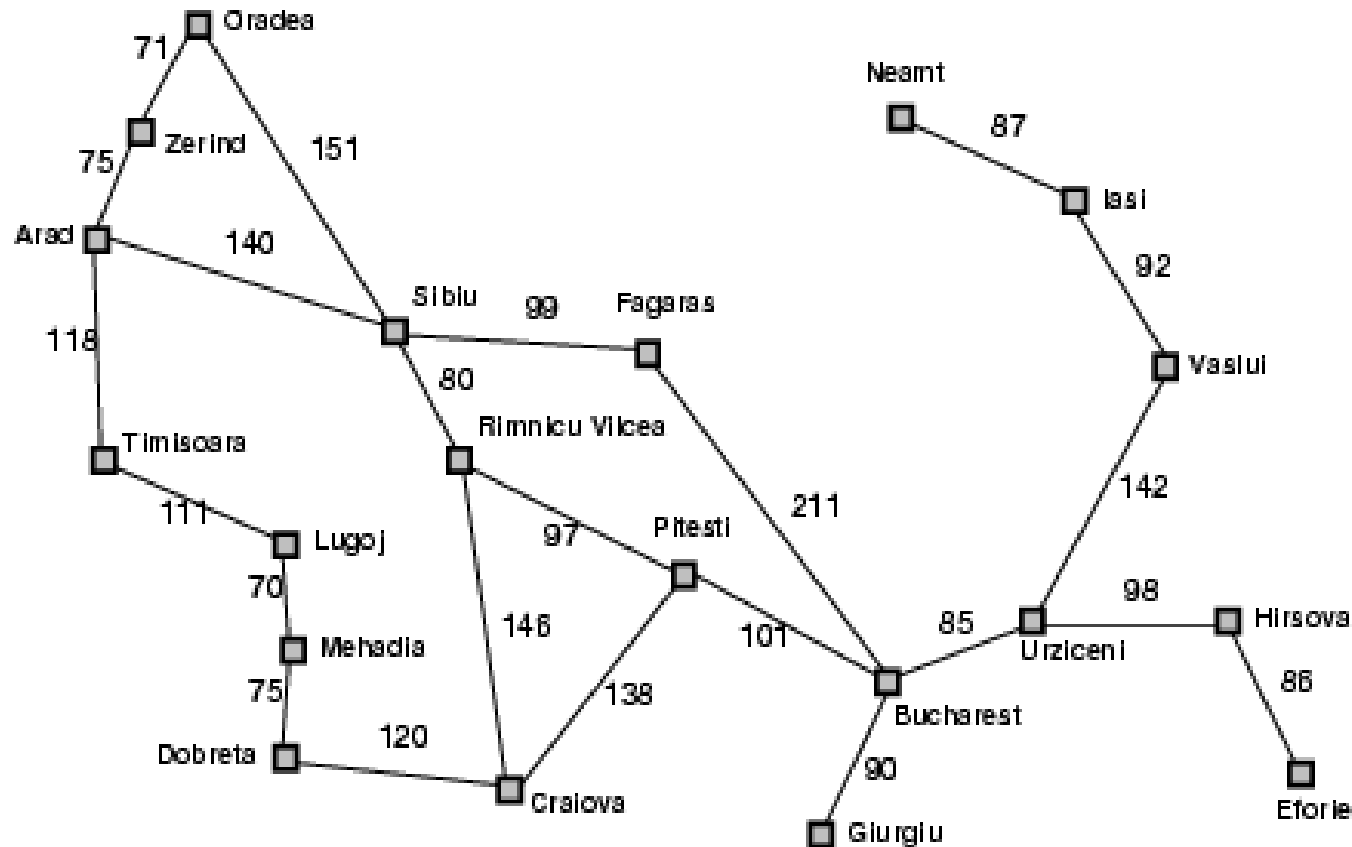
- Es un tipo de búsqueda por la mejor opción donde la función de evaluación: $f(n) = h(n)$
 - $h(n)$ = estimado del costo del camino mas barato para llegar al objetivo desde n (heurística)
 - $h(\text{nodo_objetivo}) = 0$
 - **Ejemplo:** $h(n) = h_{DLR}(n)$ = distancia en línea recta desde n hasta el objetivo.
- La búsqueda codiciosa o voraz expande el nodo en la frontera con menor $h(n)$; ósea, el que parece que está mas próximo al objetivo de acuerdo a la función heurística.



Greedy Best-First Search

(Búsqueda Voraz)

Ejemplo de búsqueda voraz en el mapa de Rumania siendo el objetivo Bucharest y heurística $h_{DLR}(n)$:



**Distancias en línea recta
hasta Bucarest**

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



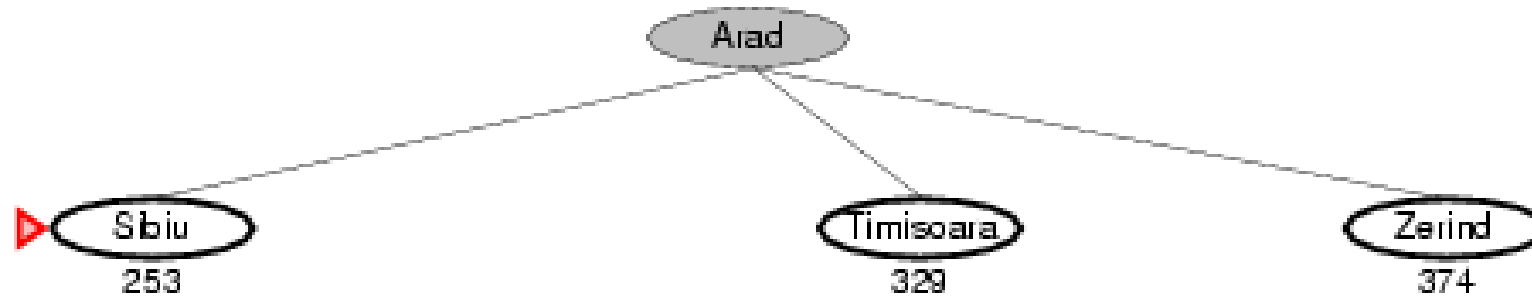
Arad
366



Greedy Best-First Search

(Búsqueda Voraz)

Ejemplo de búsqueda voraz en el mapa de Rumania siendo el objetivo Bucharest y heurística $h_{DLR}(n)$:



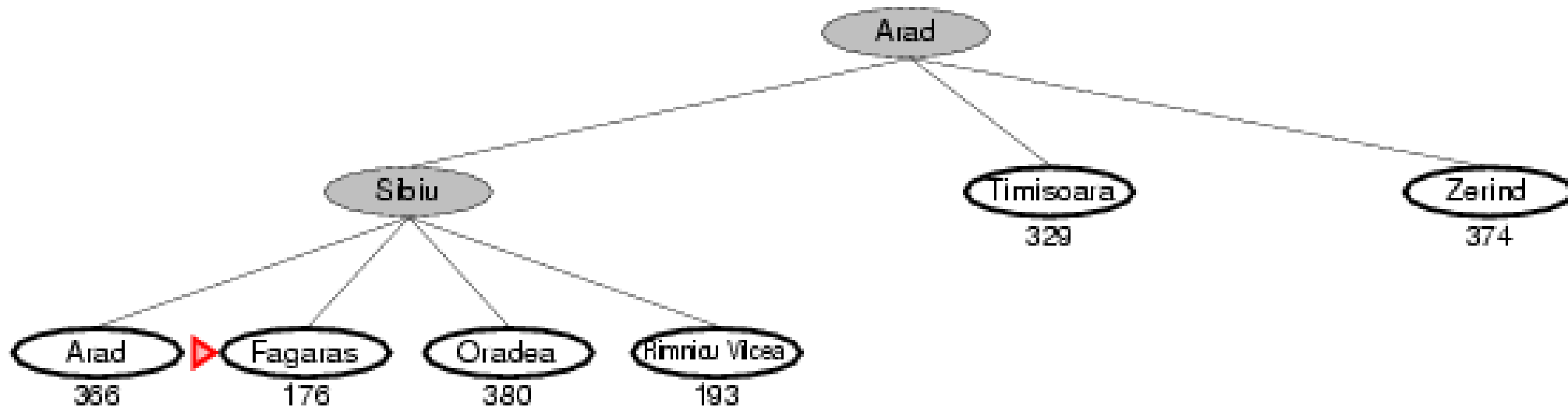
¿Cuál nodo frontera expande primero?



Greedy Best-First Search

(Búsqueda Voraz)

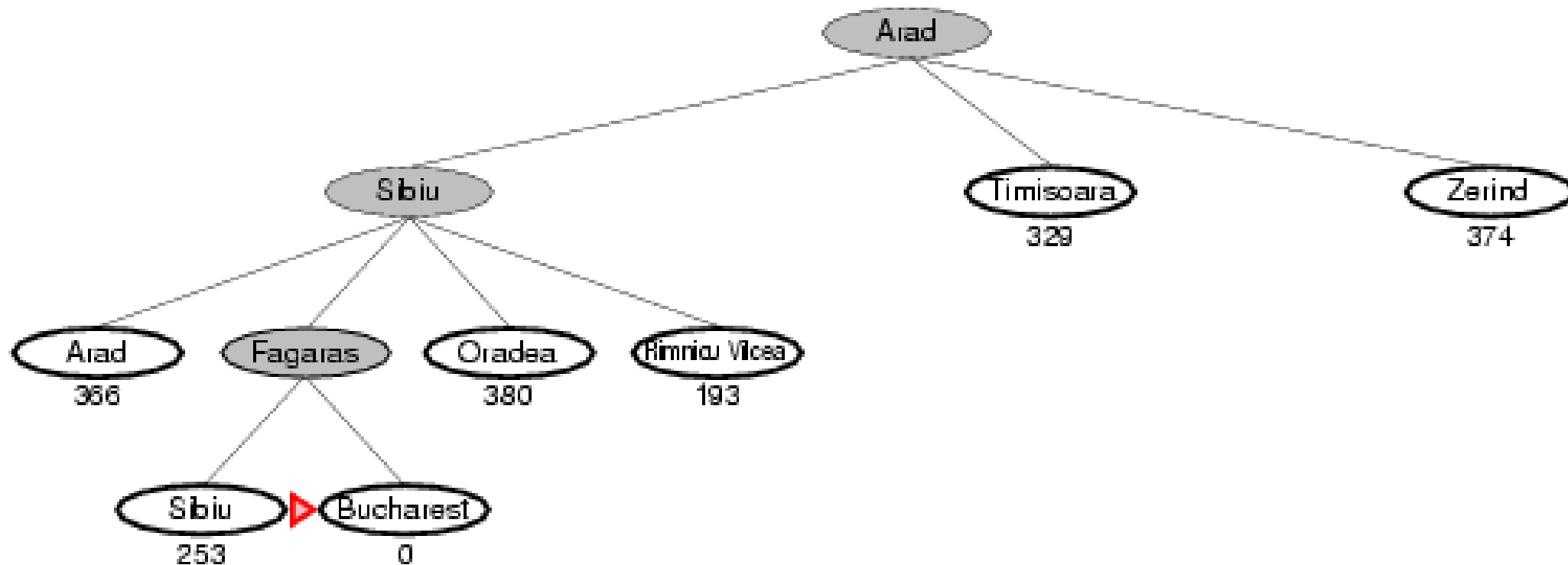
Ejemplo de búsqueda voraz en el mapa de Rumania siendo el objetivo Bucharest y heurística $h_{DLR}(n)$:



Greedy Best-First Search

(Búsqueda Voraz)

Ejemplo de búsqueda voraz en el mapa de Rumania siendo el objetivo Bucharest y heurística $h_{DLR}(n)$:



¿Óptimo? El camino via Rimnicu Vilcea y Pitesti es 32 km mas corto!



Greedy Best-First Search

(Búsqueda Voraz)

Análisis algorítmico:

- ¿Completa? **SÍ**, si chequea estados repetidos (graph-search en espacios finitos). Caso contrario (tree-search), no.
- Complejidad de tiempo: $O(b^m)$ en el peor caso, pero una buena función heurística puede llevar a una reducción substancial
- Complejidad de espacio: $O(b^m)$, mantiene todos los nodos en memoria.
- ¿Óptima? **NO**. Puede haber un camino mejor siguiendo algunas opciones peores en algunos nodos del árbol de búsqueda



A* Search

(Búsqueda A-star o A-estrella)

- Es la forma mas común de búsqueda por la mejor opción
- Función de evaluación: $f(n) = g(n) + h(n)$
 - $g(n)$ = costo del camino para alcanzar n
 - $h(n)$ = costo estimado del camino mas barato de n hasta el objetivo
 - $f(n)$ = costo total estimado del camino mas barato hasta el objetivo pasando por n



A* Search

(Búsqueda A-star o A-estrella)

Ejemplo de búsqueda A* en el mapa de Rumania siendo el objetivo Bucarest y heurística $h_{DLR}(n)$:

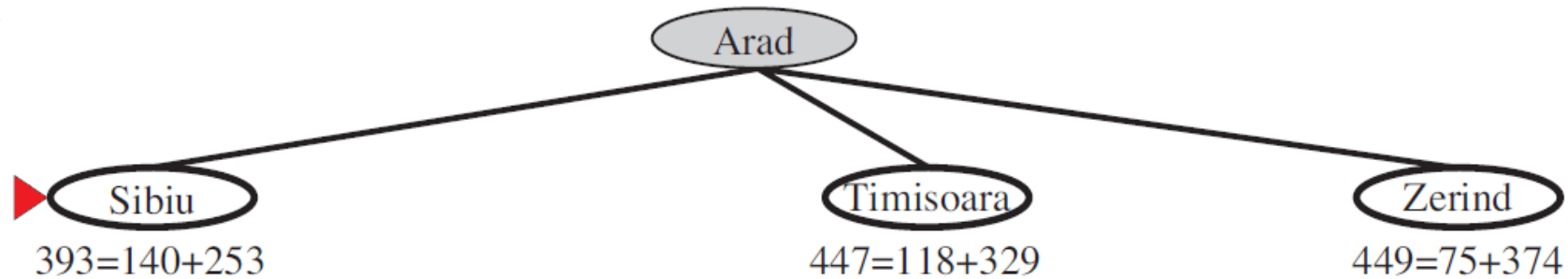
▶ Arad
 $366=0+366$



A* Search

(Búsqueda A-star o A-estrella)

Ejemplo de búsqueda A* en el mapa de Rumania siendo el objetivo Bucarest y heurística $h_{DLR}(n)$:



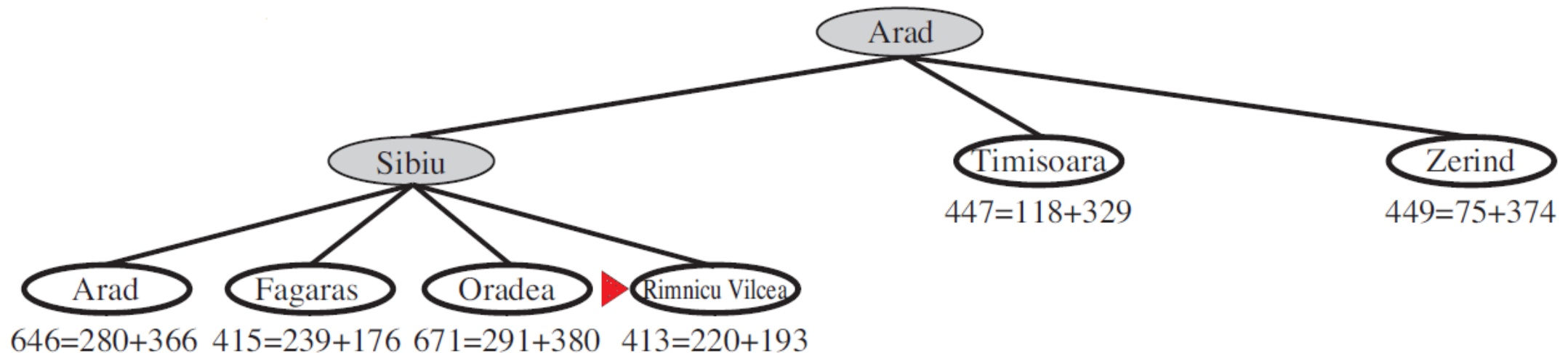
¿Cuál nodo frontera expande primero?



A* Search

(Búsqueda A-star o A-estrella)

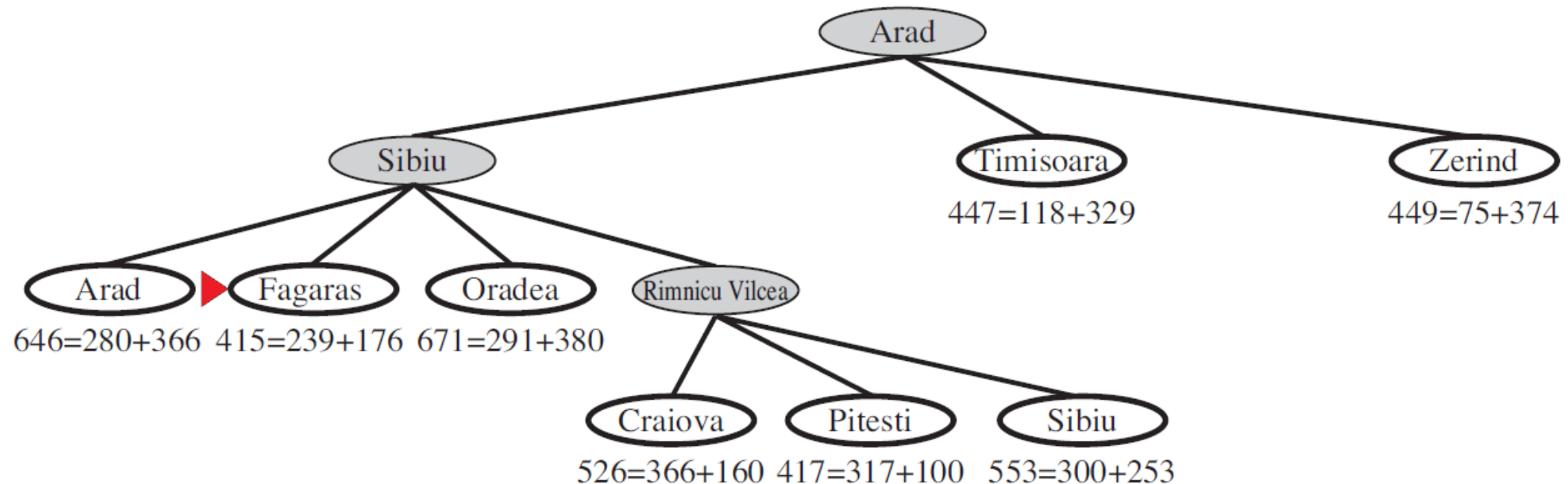
Ejemplo de búsqueda A* en el mapa de Rumania siendo el objetivo Bucarest y heurística $h_{DLR}(n)$:



A* Search

(Búsqueda A-star o A-estrella)

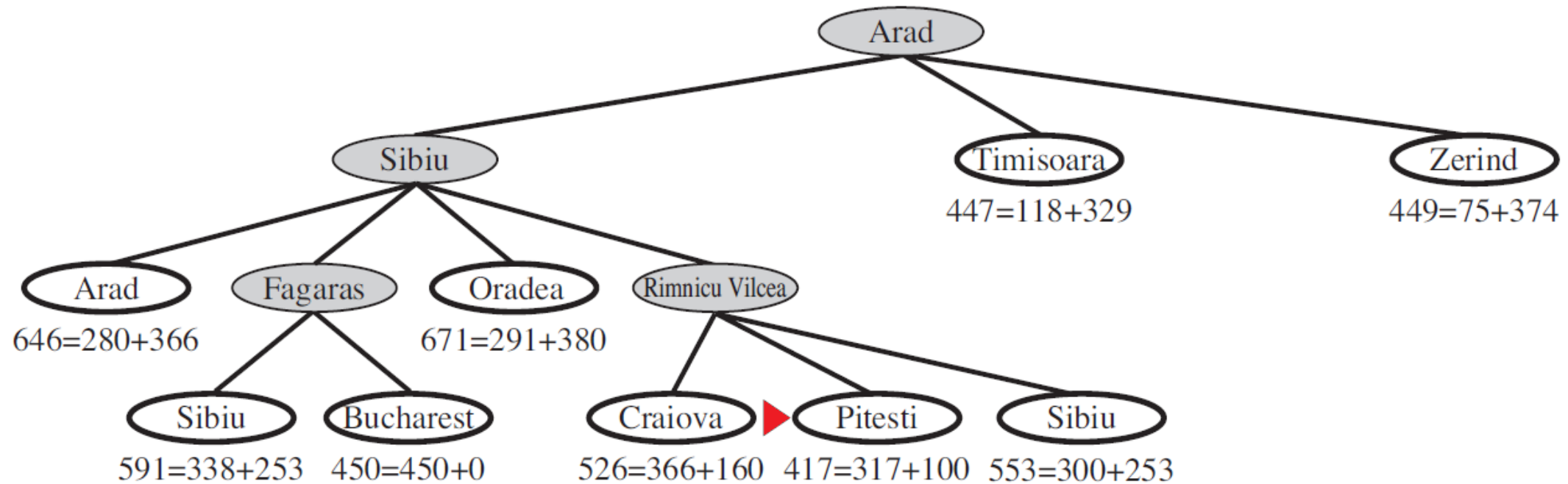
Ejemplo de búsqueda A* en el mapa de Rumania siendo el objetivo Bucarest y heurística $h_{DLR}(n)$:



A* Search

(Búsqueda A-star o A-estrella)

Ejemplo de búsqueda A* en el mapa de Rumania siendo el objetivo Bucarest y heurística $h_{DLR}(n)$:



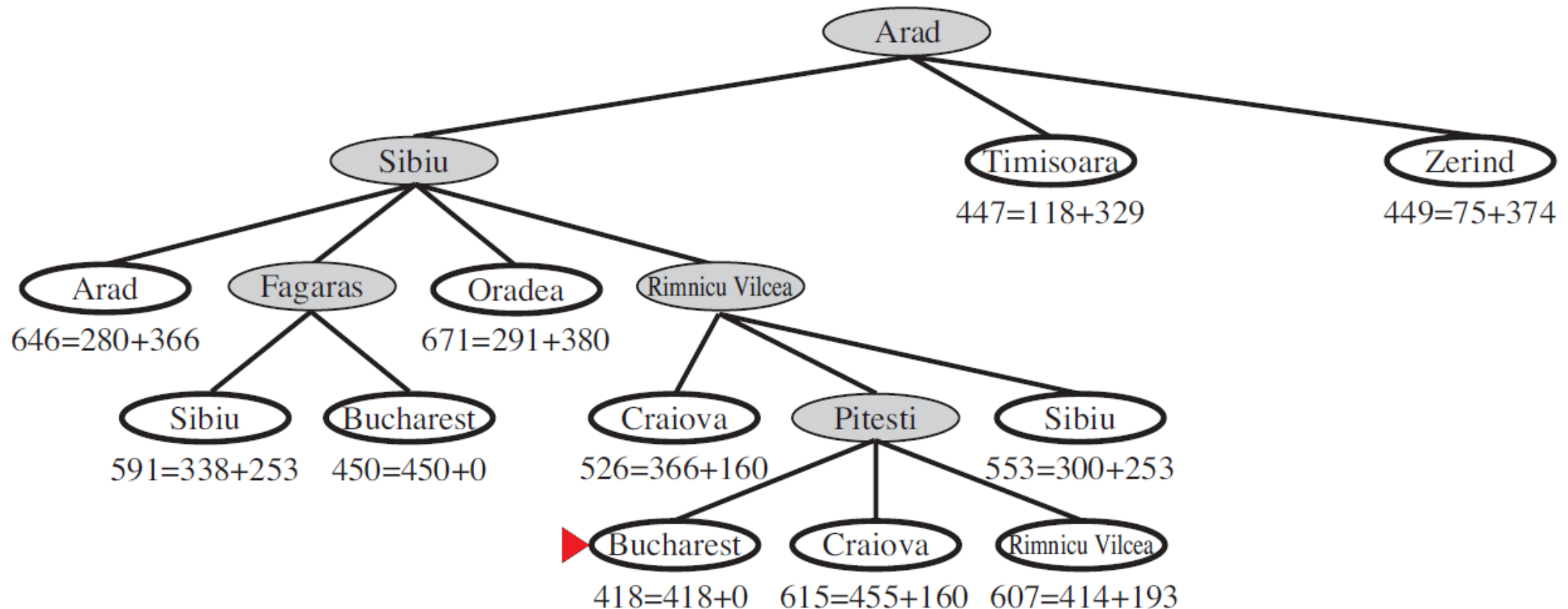
Bucarest ya se generó, pero aún no se expande:
falta explorar una solución posible de menor costo.



A* Search

(Búsqueda A-star o A-estrella)

Ejemplo de búsqueda A* en el mapa de Rumania siendo el objetivo Bucarest y heurística $h_{DLR}(n)$:



A* Search

(Búsqueda A-star o A-estrella)

Condiciones para la optimalidad: Admisibilidad

- Una heurística $h(n)$ es **admisibile** si para cada nodo n se verifica:

$$h(n) \leq h^*(n); \quad h(n_{\text{objetivo}}) = 0$$

donde $h^*(n)$ es el costo **verdadero** de alcanzar el estado objetivo a partir de n

- Una heurística admisible **nunca sobreestima** el costo de alcanzar el objetivo, es decir, la heurística siempre es optimista.
 - Ejemplo: $h_{DLR}(n)$ (distancia en línea recta nunca es mayor que distancia por las calles con curvas, obstáculos o desvíos).
- Teorema:** Si $h(n)$ es admisible, A* es optimo con búsqueda en árbol (sin memoria de estados visitados).



A* Search

(Búsqueda A-star o A-estrella)

Demostrando la optimalidad en tree-search:

- Queremos demostrar que si $h(n)$ es admisible, entonces A* expande primero una solución óptima.
- Asumimos la existencia de un camino óptimo con un costo C^* . A* expandirá el nodo objetivo cuando no exista otro nodo n en la frontera tal que $f(n) < C^*$.
- Por reducción al absurdo: asumimos que A* expande un nodo objetivo n_{objetivo} con costo total $g(n_{\text{objetivo}}) = C > C^*$ (solución sub-óptima).
- Lo anterior implica que en la frontera existe todavía un nodo n que está en el camino de la solución óptima. Para este nodo se cumple: $f(n) = g(n) + h(n)$. Como $h(n)$ es admisible, entonces $h(n) \leq h^*(n) \rightarrow g(n) + h(n) \leq g(n) + h^*(n)$.
- De lo anterior: $f(n) \leq g(n) + h^*(n) = \text{costo real total desde inicio pasando por } n = C^* \rightarrow f(n) \leq C^*$.
- O lo que es lo mismo: $C^* \geq f(n) \dots (1)$
- Por otro lado, $f(n_{\text{objetivo}}) = g(n_{\text{objetivo}}) + h(n_{\text{objetivo}}) = C + 0 = C \dots (2)$
- Se sabe que $C > C^*$. Aplicando (1): $C > C^* \geq f(n) \rightarrow C > f(n)$. Por (2), $f(n_{\text{objetivo}}) > f(n)$.
- Entonces A* debió haber expandido n antes que n_{objetivo} . Ello contradice la hipótesis inicial.



A* Search

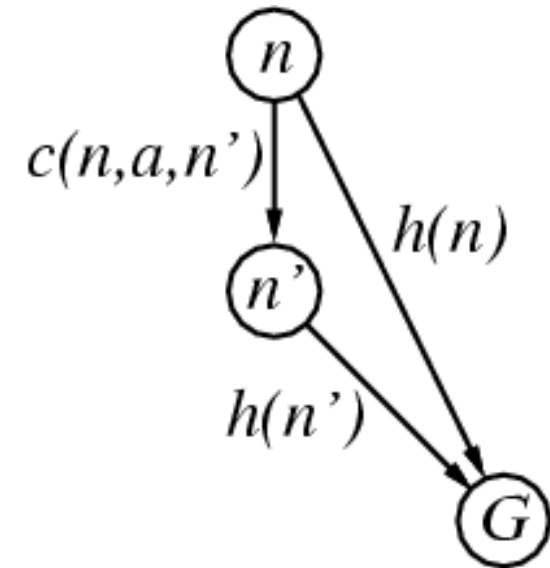
(Búsqueda A-star o A-estrella)

Condiciones para la optimalidad: Consistencia

- Una heurística $h(n)$ es **consistente** (o **monotónica**) si para cada nodo n y sucesor n' generado por acción a se verifica que:

$$h(n) \leq c(n,a,n') + h(n')$$

- Además: $h(\text{objetivo}) = 0$
- Es una forma de la desigualdad triangular (cada lado del triángulo no puede ser mayor que la suma de los otros lados).
- Si una heurística es consistente entonces también es admisible.
- Teorema:** Si $h(n)$ es consistente, A* es óptimo con búsqueda en grafo (con memoria de estados visitados).



A* Search

(Búsqueda A-star o A-estrella)

Demostrando la optimalidad en graph-search (material extra):

- Propiedades clave de la consistencia necesarias para demostración:

1. La función de evaluación es monotónica no-decreciente:

Si n' es un nodo sucesor a n : $f(n') = g(n') + h(n') = g(n) + c(n,a,n') + h(n') \dots (1)$

Dado que $h(n)$ es consistente: $h(n) \leq c(n,a,n') + h(n') \dots (2)$

Reordenando (2) y sumando $g(n)$ se obtiene: $c(n,a,n') + h(n') + g(n) \geq h(n) + g(n) \dots (3)$

Reemplazando (1) en (3): $f(n') \geq f(n)$



A* Search

(Búsqueda A-star o A-estrella)

Demostrando la optimalidad en graph-search (material extra):

- Propiedades clave de la consistencia necesarias para demostración:

2. Cuando se expande un nodo, el camino encontrado es óptimo:

Sea n un nodo que se expande en un momento determinado, con un costo $g(n)$.

Por reducción al absurdo: supongamos que existe un camino con costo menor $g'(n)$.

Entonces quedaría en la frontera un nodo tal que $g'(n) < g(n)$.

Los valores de f son: $f(n) = g(n) + h(n)$ y $f'(n) = g'(n) + h(n)$.

Como $g'(n) < g(n) \rightarrow f'(n) < f(n)$

Luego, A* debería haber expandido primero el nodo con costo $g'(n)$.

Lo anterior se contradice al supuesto inicial.



A* Search

(Búsqueda A-star o A-estrella)

Demostrando la optimalidad en graph-search (material extra):

- A partir de las 2 propiedades anteriores, se tiene:
 1. La primera vez que se expande un nodo n , el camino hace n es óptimo:
 - En particular, para el nodo objetivo (n_{objetivo}), la primera vez que se expande, el camino tiene costo $g(n) = C^*$.
 2. A* con graph-search encuentra la solución óptima:
 - A* termina al expandir n_{objetivo}
 - Como la primera expansión de n_{objetivo} corresponde a un camino óptimo (por 1), la solución que se retorna es óptima.

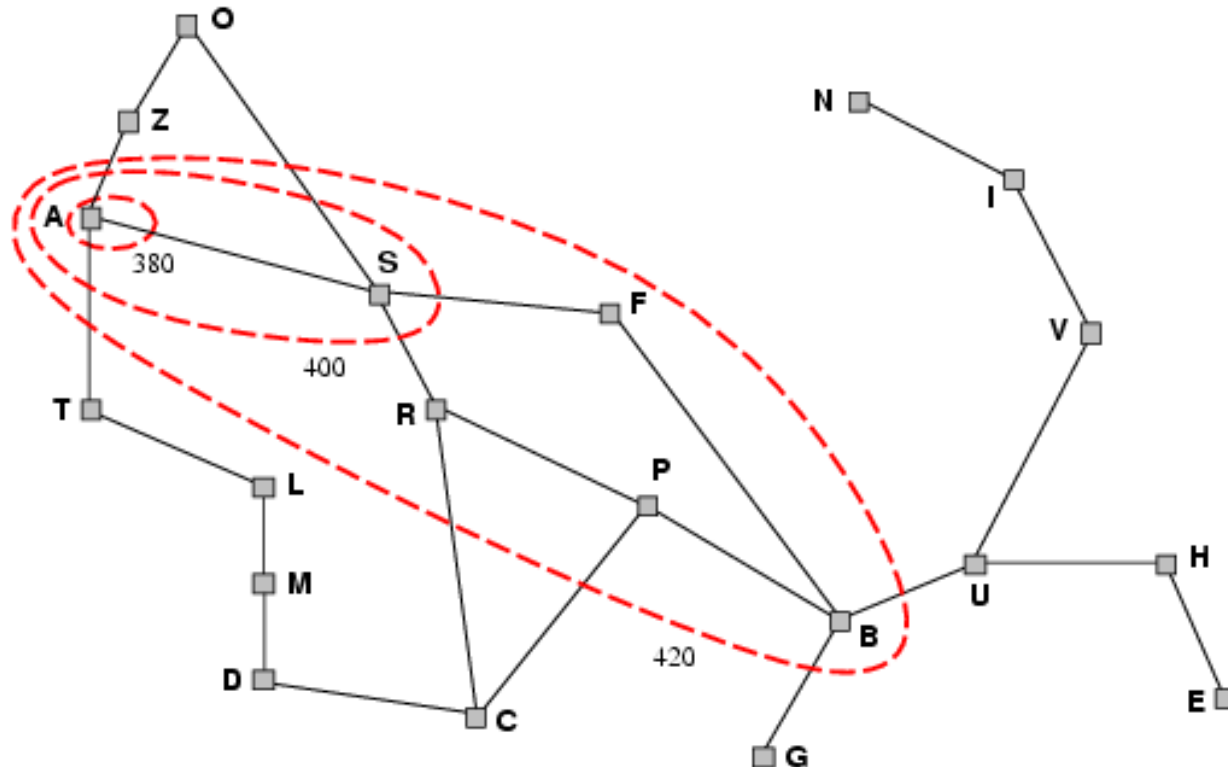


A* Search

(Búsqueda A-star o A-estrella)

Contornos de valores f en el espacio de estados que A* traza

- A* expande nodos en orden creciente de valores de f
- Gradualmente adiciona contornos de nodos
- Los estados fuera del contorno i tienen $f > f_i$, donde $f_i < f_{i+1}$
- No expande nodos con $f(n) > C^*$ (costo de la solución óptima)



Si $h(n)=0$ tenemos una **búsqueda de costo uniforme** $\Rightarrow$ círculos concéntricos.

Cuanto mejor la heurística mas direccionados son las elipses hacia el objetivo



A* Search

(Búsqueda A-star o A-estrella)

Análisis algorítmico de A*

- ¿Completa? **Sí**, a menos que exista una cantidad infinita de nodos con $f(n) < C^*$
- **Complejidad de tiempo:** Exponencial en el peor de los casos (heurística no apropiada)
- **Complejidad de espacio:** También exponencial en el peor caso ya que mantiene todos los nodos en memoria. Con frecuencia se agota el espacio antes que el tiempo.
- ¿Optima? **SI, si heurística es admisible y consistente (Graph-search)**
- **Óptimamente Eficiente:** Ningún otro algoritmo de búsqueda garantiza expandir un número menor de nodos que A* y encontrar la solución optima. Esto porque cualquier algoritmo que no expande todos los nodos con $f(n) < C^*$ corre el riesgo de omitir una solución optima.



Heurísticas

Ejemplo de heurísticas admisibles

- Para el rompecabezas de 8 piezas:
 - $h_1(n)$ = número de piezas fuera de posición
 - $h_2(n)$ = distancia “Manhattan” total (para cada pieza calcular la distancia en movidas verticales y horizontales hasta su posición objetivo)

Ejercicio

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- $h_1(S) = ?$
- $h_2(S) = ?$

¿Por qué son admisibles?



Heurísticas

Dominancia:

- Una heurística h_2 **domina** a otra heurística h_1 si para todo nodo n :

$$h_2(n) \geq h_1(n)$$

- h_2 **es mejor que** h_1 cuando h_2 **domina** h_1
 - La heurística dominante h_2 , al tener mayores valores de h en cada nodo, expande nodos más próximos del objetivo de que h_1 , significando menos nodos expandidos.
 - Como $h_2(n) \geq h_1(n) \rightarrow f_2(n) \geq f_1(n)$. Dado que h es admisible, entonces nunca se expandirán nodos tales que $f(n) \geq C^*$. Dado que $f_1(n) \leq f_2(n)$, entonces los nodos que no deban expandirse con h_1 , tampoco lo harán con h_2 ; sin embargo lo opuesto no es necesariamente cierto.

Bajo el supuesto que $f_1(n) \leq f_2(n)$, solo hay 3 escenarios posibles (la 4ta combinación es imposible):

(1) Que $f_1(n) > C^*$ y $f_2(n) > C^* \rightarrow n$ no se expande ni por f_1 ni por f_2

(2) Que $f_1(n) < C^*$ y $f_2(n) < C^* \rightarrow n$ se expande por f_1 y por f_2

(3) Que $f_1(n) < C^*$ y $f_2(n) > C^* \rightarrow n$ se expande por f_1 y pero **NO** se expande por f_2

$\Rightarrow$ La cantidad de nodos que expande f_1 es mayor a la de $f_2 \rightarrow h_2$ es mejor que h_1



Heurísticas

Como crear heurísticas admisibles: problemas relajados

- **Problema relajado:** versión simplificada de un problema, con menos restricciones.
- Toda solución óptima en el problema original es una solución en el problema relajado.
- Espacio de soluciones aumenta (cantidad de aristas) → existe potencial “atajo” al objetivo.
- *El costo de la solución óptima del problema relajado puede ser una heurística para el original.*



Heurísticas

Como crear heurísticas admisibles: problema relajado

- Por ejemplo, en el problema del rompecabezas de 8 piezas, la formulación original es:

Una pieza puede moverse del cuadrado A al cuadrado B **si y solo si:**

A es horizontalmente o verticalmente adyacente a B **y** B es blanco

7	2	4
5		6
8	3	1

Podríamos generar los siguientes problemas relajados:

- Una pieza puede moverse del cuadrado A al cuadrado B
- Una pieza puede moverse del cuadrado A al cuadrado B **si** A es adyacente a B

El costo de la solución óptima del problema (I) sería h_1 (# piezas fuera de lugar), mientras que el costo de la solución optima del problema (II) sería h_2 (distancia Manhattan)

- Es importante que el problema relajado pueda resolverse sin recurrir a búsquedas.



Heurísticas

Como crear heurísticas admisibles: problema relajado

- Las heurísticas generadas con problemas relajados son **admisibles**, ya que el costo de la solución del problema relajado nunca va ser mayor que el del problema original
- También son **consistentes** para el problema original si lo son para el problema relajado
- Si se tiene una colección de heurísticas admisibles $h_1 \dots h_m$ y ninguna domina a las demás, se puede generar una **nueva heurística h** que domina a todas:

$$h(n) = \max\{h_1(n), \dots, h_m(n)\}$$



Bibliografía



Capitulo 3.5 y 3.6 del libro:

Stuart Russell & Peter Norvig “Artificial Intelligence: A modern Approach”, Prentice Hall, Third Edition, 2010

¡Gracias!

